Mini-Pilha de Rede em Camadas

Trabalho Prático 2: implementação em C sobre soquetes UDP

Kevin Miyahara Omori
Programa de Graduação em Ciência da Computação, IME-USP

Outubro de 2026

1 Arquitetura do sistema

A mini-pilha é um programa em C (`ministack`) que implementa quatro camadas independentes: aplicação, transporte, rede e enlace. Cada instância do programa é um nó da rede, identificado por um endereço lógico de 16 bits. Os nós conversam por soquetes UDP reais, mas o enlace é simulado: o soquete UDP faz o papel do meio físico, e a camada de enlace decide, para cada frame, se ele é perdido, atrasado ou corrompido antes de chegar ao vizinho.

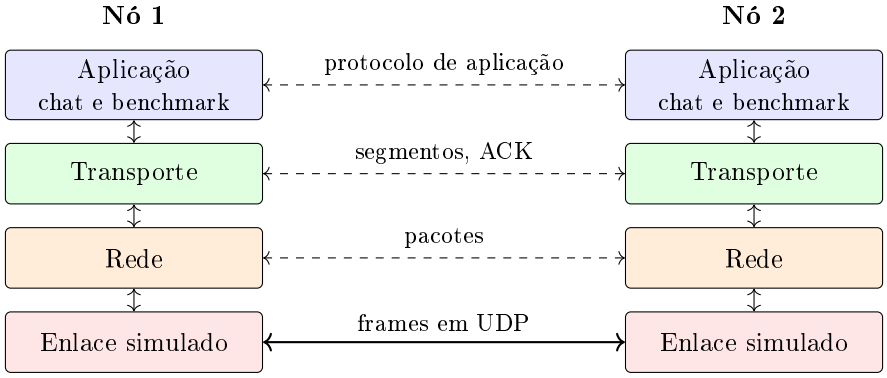


Figura 1: Pilha de dois nós. Cada camada conversa apenas com a vizinha no mesmo nó (setas cheias) e, logicamente, com a camada par do outro nó (setas tracejadas).

Cada nó é um único processo com um laço de eventos baseado em `select`: ele espera o soquete UDP, a entrada padrão (no modo chat) e o próximo prazo de qualquer camada (temporizador de retransmissão do transporte, frames retidos pelo atraso simulado, próxima mensagem agendada da aplicação). Com isso não há *threads* nem travas, e o comportamento de cada camada é determinístico dado o mesmo conjunto de eventos.

A mesma pilha suporta mais de dois nós: o endereçamento da camada de rede e a tabela de rotas estática permitem que um nó intermediário encaminhe pacotes entre dois outros, como nos testes de três nós da Seção 7.

Siglas. UDP (*User Datagram Protocol*); TTL (*time to live*, número máximo de saltos de um pacote); CRC (*cyclic redundancy check*, código de detecção de erros); ACK (*acknowledgement*, confirmação de recebimento); RTT (*round-trip time*, tempo de ida e volta); RTO (*retransmission timeout*, temporizador de retransmissão); KiB (*kibibyte*, 1024 bytes).

Arquivo	Camada	Conteúdo
`src/application.c`	Aplicação	chat com sessão; aplicação de benchmark
`src/transport.c`	Transporte	portas, numeração, ACK, retransmissão, modo não confiável
`src/network.c`	Rede	endereços, cabeçalho, rotas estáticas, TTL, encaminhamento
`src/link.c`	Enlace	frame, CRC-16, perda, atraso, jitter, corrupção; soquete UDP
`src/node.c`	Integração	opções, ligação entre camadas, laço de eventos
`src/common.c`	Apoio	relógio, leitura e escrita de inteiros em ordem de rede, trace

Tabela 1: Organização do código-fonte. Os cabeçalhos públicos ficam em `include/`.

2 Responsabilidades de cada camada

2.1 Aplicação

Duas aplicações compartilham o mesmo nó, cada uma em sua porta lógica. O **chat** (porta 1) tem protocolo próprio com controle de sessão: um nó abre a sessão com `HELLO`, o outro confirma com `HELLO_ACK`, e só depois mensagens `TEXT` são aceitas; `BYE` encerra a sessão. Texto recebido fora de uma sessão estabelecida é rejeitado e contado. O **benchmark** (porta 2) envia uma quantidade configurável de mensagens numeradas, com conteúdo determinístico, e o receptor confere ordem, duplicatas, lacunas e integridade do conteúdo; é ele que torna os experimentos mensuráveis.

2.2 Transporte

Responsável pela comunicação entre processos. Identifica origem e destino por portas lógicas, o que multiplexa várias aplicações sobre o mesmo endereço de rede. No modo confiável, numera os segmentos, mantém uma janela deslizante, confirma o recebimento com ACK cumulativo e retransmite quando o temporizador expira. No modo não confiável, envia os segmentos numerados sem ACK, e o receptor apenas contabiliza lacunas. Se uma mensagem esgota o limite de retransmissões, o transporte avisa a aplicação.

2.3 Rede

Identifica nós e entrega pacotes. Cada pacote carrega endereço de origem, de destino e um TTL. Se o destino é o próprio nó, o segmento sobe ao transporte; caso contrário, o pacote é encaminhado pelo vizinho indicado na tabela de rotas, com o TTL decrementado, e é descartado quando o TTL se esgota ou não há rota.

2.4 Enlace (simulado)

Constrói o frame (tamanho e CRC-16 de 16 bits), entrega-o ao vizinho por UDP e simula as imperfeições do meio: perda, atraso fixo, jitter e corrupção de bits, todos configuráveis por opção de linha de comando e com semente *pseudoaleatória* fixa para reprodutibilidade. No recebimento, valida o tamanho e o CRC e descarta frames inválidos. A verificação do CRC pode ser desligada, o que permite medir o valor da detecção de erros.

3 Interfaces entre camadas

A regra de separação é estrutural: cada camada só inclui o cabeçalho da camada imediatamente abaixo e chama apenas as primitivas dela. A comunicação para cima usa *callbacks*: o módulo de integração (`node.c`), o único que conhece todas as camadas, liga o enlace à rede e a rede ao transporte, e a própria aplicação registra os seus no transporte com `tp_bind`. Assim, nenhuma camada depende da implementação da que está acima. Um teste automático confere as inclusões de cada módulo (Seção 7.7).

Interface	**Primitivas (para baixo)**	***Callback* (para cima)**
Aplicação e transporte	`tp_bind(porta, on_data, on_error)`, `tp_send(porta, destino, porta_destino, dados, tamanho, confiável)`, `tp_pending()`	`tp_data_fn(origem, porta, dados, tamanho)`, `tp_error_fn(destino, porta, erro)`
Transporte e rede	`net_send(destino, segmento, tamanho)`	`net_rx_fn(origem, segmento, tamanho)`, registrado com `net_set_upper`
Rede e enlace	`link_send(vizinho, pacote, tamanho)`	`link_rx_fn(vizinho, pacote, tamanho)`, registrado com `link_set_upper`
Enlace e meio	`sendto` e `recvfrom` sobre UDP	`link_on_readable()` chamado pelo laço de eventos

Tabela 2: Interfaces entre camadas. Os contratos completos estão nos cabeçalhos de `include/`.

Além das primitivas de dados, cada camada com estado temporal expõe `*_tick(agora)` e `*_next_deadline()`, que o laço de eventos usa para acordar no momento certo. O retorno de `tp_send` distingue fila cheia, argumento inválido e fluxo já encerrado por falha, de modo que a aplicação reage a cada caso.

4 Formato dos cabeçalhos e encapsulamento

Todos os campos multi-byte usam ordem de rede (*big-endian*) e são escritos e lidos byte a byte, sem conversão direta de estruturas, o que evita dependência de alinhamento ou de ordem de bytes da máquina.

Camada	**Bytes**	**Campos (em bytes)**
Enlace	4	tamanho do pacote (2); CRC-16 (2), calculado sobre o cabeçalho, com o campo do CRC zerado, e o pacote; polinômio 0x1021, valor inicial 0xFFFF
Rede	9	versão (1); TTL (1); origem (2); destino (2); protocolo (1); tamanho do segmento (2)
Transporte	13	porta de origem (2); porta de destino (2); número de sequência (4); número de confirmação (4); flags (1): `DATA` 0x01, `ACK` 0x02, `REL` 0x04
Chat	1 + texto	tipo (1): `HELLO` 1, `HELLO_ACK` 2, `TEXT` 3, `BYE` 4; texto de até 200 bytes
Benchmark	5 + padrão	tipo (1) igual a 10; identificador da mensagem (4); bytes de padrão: XOR dos 4 bytes do identificador, mais o índice do byte, módulo 256

Tabela 3: Formato dos cabeçalhos de cada camada e das mensagens das aplicações.

O número de confirmação de um ACK indica o próximo número de sequência esperado (ACK cumulativo). A flag `REL` marca os segmentos do modo confiável, que são confirmados e ordenados; sem ela o segmento segue o caminho não confiável.

O encapsulamento é progressivo: cada camada recebe os dados da superior como carga útil opaca e acrescenta o próprio cabeçalho; no receptor, cada camada valida e remove o seu cabeçalho e entrega o restante à de cima. A Figura 2 mostra o resultado para a mensagem de chat "ola bob" e o trecho abaixo reproduz o trace real emitido com a opção `-v` (comando `python3 scripts/demo_chat.py`), com os tempos omitidos. As mensagens do programa são ASCII, sem acentos, para não depender da codificação do terminal.

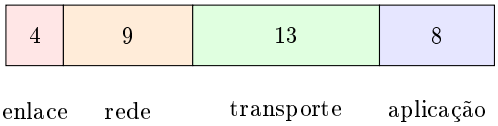


Figura 2: Frame de 34 bytes que transporta a mensagem de chat de 8 bytes (1 de tipo e 7 de texto): 4 + 9 + 13 + 8.

```
APP SEND chat TEXT "ola bob"  (1 B de tipo + 7 B de texto) -> no 2 porta 1
TRP SEND sport=1 dport=1 seq=1 ack=0 flags=DATA|REL  cabecalho 13 B + carga 8 B
NET SEND src=1 dst=2 ttl=16 proto=1 len=21  cabecalho 9 B + segmento 21 B, proximo salto 2
LNK SEND len=30 crc=0xa597  cabecalho 4 B + carga 30 B = frame 34 B -> vizinho 2
  ... no receptor, o caminho inverso (o exemplo abaixo e o recebimento de um HELLO_ACK):
LNK RECV frame 30 B, CRC ok -> pacote de 26 B para a camada de rede
NET RECV src=2 dst=1 ttl=16 proto=1 -> segmento de 17 B para o transporte
TRP RECV sport=1 dport=1 seq=0 ack=0 flags=DATA|REL  carga 4 B
APP RECV chat HELLO_ACK "bob" de 2
```

5 Decisões de projeto

Soquetes UDP como meio físico. UDP preserva a fronteira dos datagramas e não acrescenta confiabilidade, o que deixa toda a confiabilidade a cargo do transporte implementado. Cada frame é exatamente um datagrama.

Janela deslizante com ACK cumulativo (*go-back-N*). É a forma mais simples de manter vários segmentos em voo e ainda assim ter retransmissão correta: o receptor só aceita o segmento esperado e o emissor mantém um único temporizador, para o segmento mais antigo sem confirmação. Em contrapartida, uma perda faz a janela inteira ser reenviada, custo medido na Seção 7.1. A janela 1 equivale a *stop-and-wait*.

Temporizador fixo. O tempo de retransmissão é configurável e constante, sem *backoff* nem estimativa adaptativa do RTT. Isso mantém os experimentos interpretáveis; o RTT é apenas medido (amostras só de segmentos não retransmitidos).

Falha explícita. Após um número configurável de retransmissões do mesmo segmento, o fluxo é encerrado e a aplicação é avisada, em vez de tentar para sempre.

CRC-16 no enlace. A detecção de erros fica no enlace, como em redes reais, e é verificada a cada salto. Frames inválidos são descartados e tratados pelo transporte como perda.

Aplicação de benchmark com conteúdo verificável. O padrão de bytes depende do identificador da mensagem, então o receptor consegue distinguir uma mensagem perdida de uma corrompida que atravessou as camadas.

Roteamento estático com TTL. Dispensa algoritmos de roteamento, mas mantém o encaminhamento entre nós e protege contra laços.

Reprodutibilidade. Perda, atraso, jitter e corrupção usam gerador pseudoaleatório com semente informada na linha de comando; os experimentos e as figuras são gerados por scripts a partir de arquivos CSV.

6 Metodologia experimental

Os experimentos rodam dois nós (emissor e receptor) em processos separados na mesma máquina, comunicando-se pela interface de *loopback*, em Linux com um único núcleo de CPU. O emissor executa o benchmark, que envia N mensagens e termina quando todas foram confirmadas (modo confiável) ou enviadas (modo não confiável). A perda, o atraso e a corrupção são aplicados por cada nó aos frames que ele envia, portanto afetam tanto os dados quanto os ACKs.

Métricas. *Tempo de conclusão*: do envio da primeira mensagem à confirmação da última. *Taxa de retransmissão*: retransmissões divididas pelo total de segmentos de dados transmitidos, incluindo as retransmissões. *Mensagens entregues*: mensagens distintas e íntegras vistas pela aplicação receptora. *Overhead*: bytes enviados ao enlace (nos dois sentidos) divididos pelos bytes entregues pela aplicação ao transporte. O RTT é medido pelo transporte entre o envio e o ACK de segmentos não retransmitidos.

Parâmetros-base. $N = 100$ mensagens de 100 bytes, janela 4, temporizador de 60 ms, atraso de 5 ms por frame, três sementes (1, 2 e 3). Cada experimento varia um fator e mantém os demais fixos; os resultados são média e desvio padrão amostral das três repetições. Repetir uma execução com a mesma semente reproduz a sequência de perdas, mas não a ordem exata dos eventos no tempo. Refazendo todos os experimentos do zero (140 execuções em 66 configurações), as contagens (retransmissões, mensagens entregues, bytes no enlace, frames rejeitados pelo CRC e mensagens adulteradas) coincidiram exatamente com as dos arquivos de `results/`, e os tempos médios diferiram cerca de 1% ou menos nos experimentos de perda, de corrupção, de janela e perda e no modo confiável da comparação com o não confiável (desvio máximo observado de 1,0%); nos experimentos de atraso e de varredura de janela, os maiores desvios foram de 4,3% (atraso de 20 ms, janela 1) e 5,4% (janela 2). Por isso os tempos são valores aproximados, e o que os experimentos estabelecem são as relações entre eles.

Experimento	**Fator variado**	**Fixos**
Perda de pacotes	perda de 0, 5, 10, 20 e 30%	modo confiável, janela 4
Atraso na rede	atraso de 0, 5, 10, 20 e 50 ms por salto; janela 1, 4 e 16	$N = 50$; temporizador $4d + 60$ ms
Corrupção de dados	corrupção de 0, 5, 10 e 20%, com e sem verificação do CRC	atraso de 2 ms
Overhead	mensagens de 5 a 1000 bytes, confiável e não confiável	$N = 50$, sem perda
Confiável e não confiável	perda de 0, 10, 20 e 30% nos dois modos	atraso de 5 ms
Janela e perda	janela 1, 4 e 16 com perda de 0, 10 e 30%	$N = 60$; atraso de 20 ms (5 ms com 30%)
Varredura de janela	janela de 1 a 64 (potências de 2)	perda de 5%, atraso de 1 ms, $N = 300$, temporizador de 80 ms

Tabela 4: Resumo dos experimentos.

Todos os dados são reproduzíveis com três comandos:

```
make test                            # testes automatizados
python3 scripts/run_experiments.py   # grava os CSVs em results/
python3 scripts/make_assets.py       # gera as tabelas e figuras
```

7 Resultados e análise

7.1 Impacto da perda de pacotes

Perda	Tempo (ms)	Retransmissões	Taxa de retransmissão (%)	Vazão (KiB/s)	Completas
0%	260 ± 0	0 ± 0	0,0 ± 0,0	37,6	3/3
5%	590 ± 133	20 ± 8	16,4 ± 5,6	17,1	3/3
10%	1006 ± 139	45 ± 9	30,7 ± 4,7	9,8	3/3
20%	2460 ± 258	131 ± 14	56,6 ± 2,5	4,0	3/3
30%	3413 ± 309	194 ± 19	65,9 ± 2,3	2,9	3/3

Tabela 5: Perda de frames, modo confiável (média e desvio de 3 execuções).

Todas as execuções terminaram com as 100 mensagens entregues e em ordem (os testes de integração verificam também que não há duplicatas na aplicação): o ACK e a retransmissão realmente entregam uma comunicação confiável sobre um meio que perde frames. O preço é o tempo. Sem perdas, a transferência leva 260 ms, coerente com $\lceil N/\text{janela} \rceil = 25$ rodadas de um RTT de cerca de 10,4 ms. Com 30% de perda, o tempo sobe para 3413 ms (13 vezes) e a vazão cai de 37,6 para 2,9 KiB/s. A taxa de retransmissão cresce mais depressa que a perda: com 10% de perda de frames, 30,7% dos segmentos transmitidos são retransmissões, e com 30% chega a 65,9% (cerca de 2,9 transmissões por mensagem). Duas causas se somam: tanto o dado quanto o ACK podem se perder (com janela 1, em que não há reenvio de segmentos já recebidos, a taxa com 30% de perda cai para 54,8%, próxima dos 51% previstos por $1 - 0{,}7^2$), e o *go-back-N* reenvia toda a janela a cada expiração, inclusive segmentos que já haviam chegado (66,9% com janela 4 e 84,0% com janela 16, Tabela 7).

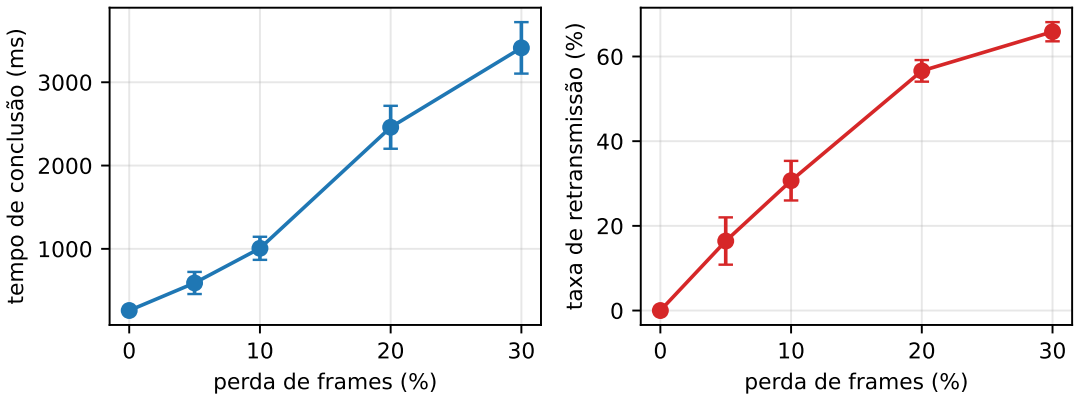


Figura 3: Tempo de conclusão e taxa de retransmissão em função da perda (barras: desvio padrão).

7.2 Impacto do atraso na rede

Atraso por salto (ms)	Tempo, janela 1 (ms)	Tempo, janela 4 (ms)	Tempo, janela 16 (ms)	RTT medido (ms)	2× atraso (ms)
0	1	1	1	0,0	0
5	520	135	42	10,4	10
10	1023	273	82	20,5	20
20	2027	526	163	40,5	40
50	5023	1306	402	100,5	100

Tabela 6: Tempo de conclusão de 50 mensagens, por atraso e tamanho de janela.

O tempo de conclusão segue de perto o modelo $\lceil N/w \rceil \times \mathrm{RTT}$. Com 50 ms de atraso por salto e RTT medido de 100,5 ms, a janela 1 leva 5023 ms (modelo: $50 \times 100{,}5 = 5025$ ms), a janela 4 leva 1306 ms (modelo: $13 \times 100{,}5 = 1306{,}5$ ms) e a janela 16 leva 402 ms (modelo: $4 \times 100{,}5 = 402$ ms). O RTT medido também acompanha o dobro do atraso configurado (10,4 ms para 10 ms, 100,5 ms para 100 ms; o excedente vem do processamento). A janela maior compensa o atraso mantendo mais segmentos em voo, mas com perdas o ganho diminui (Tabela 7): com 20 ms de atraso, a janela 16 é 15,0 vezes mais rápida que a janela 1 sem perdas e apenas 3,7 vezes mais rápida com 10% de perda.

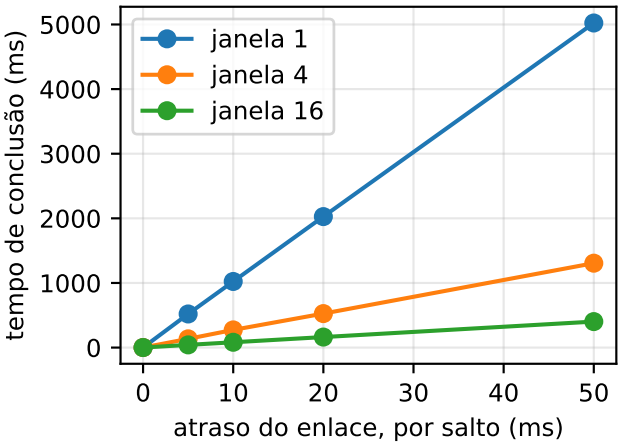


Figura 4: Tempo de conclusão em função do atraso, para três tamanhos de janela.

7.3 Taxa de retransmissão e tamanho da janela

Os valores da Tabela 5 e da Figura 3 já mostram a taxa de retransmissão em função da perda. A taxa também reage à corrupção quando o CRC está ativo, pois um frame rejeitado equivale a um frame perdido: com 20% de corrupção, as retransmissões sobem de zero para 96 por 100 mensagens (Tabela 10). A Tabela 7 separa o efeito da janela. Sem perdas, o ganho da janela segue o modelo (4,0 e 15,0 vezes, como previsto para 60 mensagens). Com perdas, o ganho encolhe e a taxa de retransmissão cresce com a janela, porque o *go-back-N* reenvia mais segmentos a cada expiração: a janela maior termina antes quando o atraso é grande, mas ao custo de bem mais tráfego redundante.

Perda	Atraso (ms)	Janela	Tempo (ms)	Ganho sobre a janela 1	Taxa de retransmissão (%)
0%	20	1	2434 ± 3	1,0	0,0 ± 0,0
0%	20	4	608 ± 1	4,0	0,0 ± 0,0
0%	20	16	163 ± 0	15,0	0,0 ± 0,0
10%	20	1	4771 ± 293	1,0	21,7 ± 2,1
10%	20	4	1767 ± 505	2,7	31,0 ± 9,5
10%	20	16	1293 ± 363	3,7	60,7 ± 8,8
30%	5	1	5037 ± 693	1,0	54,8 ± 3,9
30%	5	4	2104 ± 298	2,4	66,9 ± 3,3
30%	5	16	1486 ± 227	3,4	84,0 ± 2,0

Tabela 7: Efeito do tamanho da janela com e sem perdas (60 mensagens, 3 execuções; atraso de 20 ms, e de 5 ms na linha de 30% de perda).

A Figura 5 mostra que esse ganho tem limite. Com 5% de perda e atraso de 1 ms (300 mensagens), o tempo médio cai de 3703 ms com janela 1 para 1372 ms com janela 8 e não melhora depois (de 1347 a 1524 ms nas janelas de 16 a 64, dentro da variabilidade), enquanto a taxa de retransmissão sobe de 11,1% para 75,5%. O contador de expirações do temporizador explica o patamar: o tempo médio fica próximo de expirações × RTO (16,0 expirações × 80 ms = 1280 ms para a janela 16, contra 1347 ms medidos), e o número de expirações fica entre 15,7 e 19,0 a partir da janela 2. Cada perda de dado custa um temporizador inteiro, pois não há retransmissão antecipada por ACKs duplicados, e uma janela maior não reduz o número de perdas. Com janela 1, as 37,7 expirações também incluem ACKs perdidos, o que sugere que nas janelas maiores o ACK cumulativo seguinte mascara esse tipo de perda. Nessas condições, uma janela moderada (4 a 8) basta, e janelas maiores só aumentam o tráfego redundante.

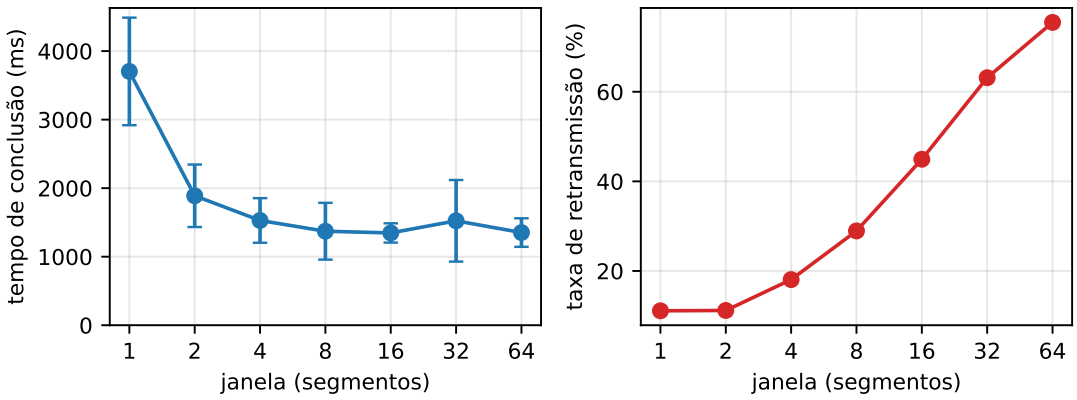


Figura 5: Tempo de conclusão e taxa de retransmissão em função da janela, com 5% de perda e 1 ms de atraso (300 mensagens, 3 execuções).

Com atraso e sem perda a taxa é zero, desde que o temporizador seja maior que o RTT; por isso, no experimento de atraso, o temporizador acompanha o atraso ($4d + 60$ ms).

7.4 Overhead introduzido pelas camadas

Mensagem (B)	Confiável, medido	Confiável, previsto	Não confiável, medido	Não confiável, previsto
5	11,4	11,4	6,2	6,2
10	6,2	6,2	3,6	3,6
50	2,04	2,04	1,52	1,52
100	1,52	1,52	1,26	1,26
250	1,208	1,208	1,104	1,104
500	1,104	1,104	1,052	1,052
1000	1,052	1,052	1,026	1,026

Tabela 8: Overhead: bytes no enlace por byte entregue pela aplicação.

Cada frame de dados carrega 26 bytes de cabeçalhos (13 do transporte, 9 da rede e 4 do enlace), e cada ACK é um frame de 26 bytes sem carga útil. O modo confiável paga, portanto, 52 bytes por mensagem, e o não confiável, 26: o overhead é $(S + 52)/S$ e $(S + 26)/S$ para mensagens de S bytes. Os valores medidos coincidem exatamente com essas fórmulas. O custo é alto para mensagens pequenas (11,4 vezes para 5 bytes, no modo confiável) e tende a 1 para mensagens grandes (1,052 para 1000 bytes). O transporte responde por metade dos 26 bytes, a rede por 35% e o enlace por 15%. Os cabeçalhos IP e UDP reais, que ficam abaixo do meio simulado (28 bytes por datagrama), não entram na conta.

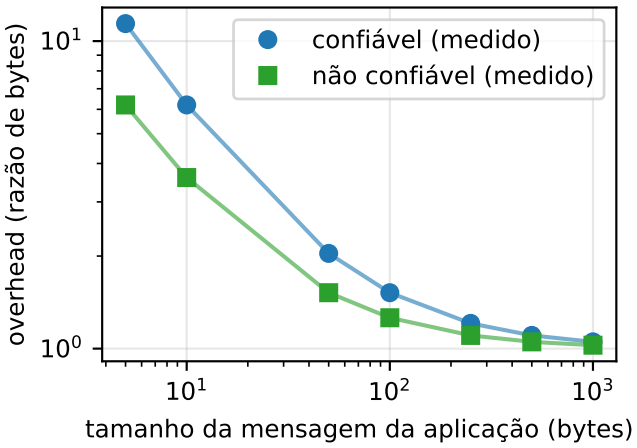


Figura 6: Overhead medido (pontos) e previsto (linhas) em função do tamanho da mensagem.

7.5 Comunicação confiável e não confiável

Perda	Transporte	Entregues (%)	Tempo (ms)	Bytes no enlace (KiB)	Retransmissões
0%	confiável	100,0 ± 0,0	260 ± 1	14,8 ± 0,0	0 ± 0
0%	não confiável	100,0 ± 0,0	n.a.	12,3 ± 0,0	0 ± 0
10%	confiável	100,0 ± 0,0	1006 ± 139	21,1 ± 1,4	45 ± 9
10%	não confiável	89,7 ± 3,2	n.a.	12,3 ± 0,0	0 ± 0
20%	confiável	100,0 ± 0,0	2463 ± 255	32,9 ± 1,8	131 ± 14
20%	não confiável	75,3 ± 5,7	n.a.	12,3 ± 0,0	0 ± 0
30%	confiável	100,0 ± 0,0	3430 ± 328	41,3 ± 2,5	194 ± 19
30%	não confiável	67,7 ± 6,0	n.a.	12,3 ± 0,0	0 ± 0

Tabela 9: Transporte confiável e não confiável sob perda (100 mensagens, 3 execuções).

O modo não confiável entrega aproximadamente a fração $1 - p$ das mensagens (89,7%, 75,3% e 67,7% para p igual a 10, 20 e 30%, contra os 90, 80 e 70% esperados; a diferença está dentro da variabilidade de três execuções de 100 mensagens), e o receptor detecta as lacunas pela numeração. O modo confiável entrega 100% em todos os casos, ao custo de mais bytes no enlace (14,8 KiB sem perdas, contra 12,3 KiB, por causa dos ACKs; 41,3 KiB com 30% de perda) e de tempo. O tempo do modo não confiável não é comparável, pois o emissor termina assim que envia, sem esperar confirmação. Os tempos do modo confiável repetem a configuração da Seção 7.1 em execuções independentes e por isso diferem em menos de 1% dos de lá (por exemplo, 3430 ms contra 3413 ms com 30% de perda).

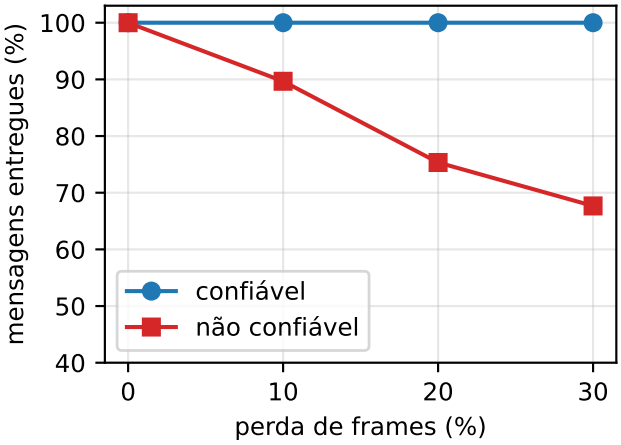


Figura 7: Mensagens entregues em função da perda, nos dois modos.

7.6 Detecção de erros no enlace

Corrupção	Enlace	Íntegras (%)	Adulteradas na app	Frames rejeitados (CRC)	Retransmissões
0%	com CRC	100,0 ± 0,0	0,0 ± 0,0	0,0 ± 0,0	0,0 ± 0,0
0%	sem CRC	100,0 ± 0,0	0,0 ± 0,0	0,0 ± 0,0	0,0 ± 0,0
5%	com CRC	100,0 ± 0,0	0,0 ± 0,0	12,0 ± 6,1	24,0 ± 12,0
5%	sem CRC	94,3 ± 3,1	5,7 ± 3,1	0,0 ± 0,0	6,7 ± 4,6
10%	com CRC	100,0 ± 0,0	0,0 ± 0,0	26,0 ± 8,2	47,7 ± 17,0
10%	sem CRC	91,3 ± 2,3	8,7 ± 2,3	0,0 ± 0,0	5,3 ± 4,6
20%	com CRC	100,0 ± 0,0	0,0 ± 0,0	66,3 ± 15,6	96,0 ± 26,2
20%	sem CRC	84,0 ± 5,3	16,0 ± 5,3	0,0 ± 0,0	19,0 ± 6,6

Tabela 10: Corrupção de bits nos frames, com e sem verificação do CRC-16.

Com o CRC ativo, nenhuma mensagem adulterada chega à aplicação em nenhum nível de corrupção: os frames inválidos são rejeitados (66 por execução com 20% de corrupção) e o transporte os recupera por retransmissão, de modo que as 100 mensagens chegam íntegras. Sem o CRC, 5,7%, 8,7% e 16,0% das mensagens chegam adulteradas com 5, 10 e 20% de corrupção. O ponto central é que o transporte confiável não ajuda nesse caso: o segmento corrompido é aceito, entregue e confirmado, então não há o que retransmitir. A confiabilidade fim a fim depende de a camada inferior detectar o erro.

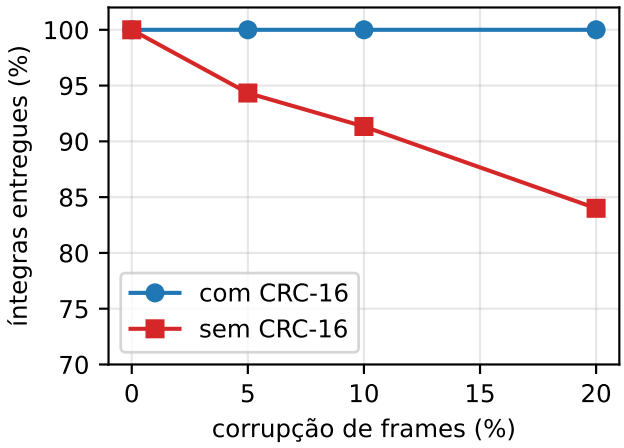


Figura 8: Mensagens íntegras entregues, com e sem CRC-16.

7.7 Validação funcional

Os testes automatizados (`make test`) somam 98 verificações: 13 de codec e robustez e 85 de integração entre processos.

Os de codec conferem o CRC contra o vetor padrão (0x29B1 para "123456789"), a ida e volta dos três cabeçalhos, a detecção de qualquer inversão de um bit em um frame cujo pacote tem 100 bytes e a robustez diante de 200 mil entradas aleatórias nas quatro camadas.

Os de integração cobrem a entrega confiável sem perdas e com 25% de perda, o modo não confiável, a corrupção com e sem CRC, o encaminhamento em três nós, a expiração do TTL, a multiplexação do chat e do benchmark no mesmo nó, a regra de separação das camadas, a validação dos argumentos e dos códigos de saída (inclusive a desistência quando o destino não existe), o intervalo entre mensagens, a rejeição de texto de chat fora de uma sessão, a rejeição de mensagem de benchmark com identificador adulterado, o descarte de frames de origem desconhecida, a fila de atraso limitada a 2048 frames, o número exato de retransmissões antes de desistir, o efeito do jitter, as estatísticas exatas do receptor (lacunas e mensagens puladas), a demonstração automática do encapsulamento (o trace da Seção 4) e as relações medidas neste relatório: a fórmula do overhead, o modelo $\lceil N/w \rceil \times \mathrm{RTT}$ (inclusive para janelas acima de 64) e o efeito da janela sob perda.

O *script* `fuzz_protocol.py` (`make fuzz`) injeta, de forma determinística, frames com CRC válido e conteúdo aleatório ou mutado em todas as camadas, em três papéis do nó (receptor com encaminhamento, iniciador de chat e emissor de benchmark), sob AddressSanitizer e UBSan.

O `conformance.py` confere, com um par escrito em Python que segue apenas o formato documentado, os campos dos três cabeçalhos, o CRC, o *go-back-N*, o reinício do temporizador, a semântica do ACK cumulativo (inclusive duplicados e inválidos), a amostragem de RTT sem retransmissões e o encaminhamento com TTL decrementado.

A suíte também foi executada sob AddressSanitizer e UBSan (`make sanitize`), sem erros.

8 Limitações e melhorias futuras

8.1 Limitações

- Sem fragmentação: cada mensagem da aplicação cabe em um segmento de até 1000 bytes.
- O *go-back-N* reenvia a janela inteira, o que desperdiça banda sob perda (Seção 7.3), e o receptor não guarda segmentos fora de ordem. Sem retransmissão antecipada por ACKs duplicados, cada perda de dado custa uma expiração inteira do temporizador, e janelas grandes aumentam o tráfego redundante sem reduzir o tempo.
- O temporizador de retransmissão é fixo e não há controle de congestionamento, nem de fluxo além do tamanho da janela.
- O CRC-16 detecta todos os erros de um bit e a maioria dos erros em rajada, mas não é infalível: erros aleatórios de vários bits passam sem detecção com probabilidade da ordem de 2^{-16}.
- Cada nó mantém no máximo 16 fluxos de transporte, e os fluxos não expiram: um vizinho configurado pode esgotar a tabela variando as portas de origem, e os segmentos de novos fluxos passam a ser descartados.
- As rotas são estáticas, definidas na linha de comando; não há descoberta de vizinhos nem roteamento dinâmico, e só há IPv4.
- As imperfeições são aplicadas por nó ao que ele envia; não há reordenação dirigida de pacotes (a reordenação só ocorre como efeito do jitter).
- Os experimentos rodam em *loopback* com um núcleo de CPU; os tempos absolutos dependem da máquina, embora as tendências e as relações (como $\lceil N/w \rceil \times \mathrm{RTT}$) não dependam.

8.2 Melhorias futuras

- Retransmissão seletiva com buffer de recepção, para retransmitir apenas o que se perdeu, e retransmissão antecipada após ACKs duplicados.
- Estimativa adaptativa do RTT para o temporizador, com *backoff* exponencial.
- Fragmentação e remontagem de mensagens maiores que o segmento.
- Controle simples de congestionamento (por exemplo, janela que cresce e encolhe conforme perdas).
- Roteamento dinâmico por vetor de distâncias entre os nós.
- Suporte a IPv6 e interface gráfica para visualizar o encapsulamento.